Weekly Report: [W05]

General Information

- **Group ID:** 52
- **Group Name:** Group 52
- **Class:** 25A01
- **Project Name:** Super Mario Game
- **Date range:** 2026-07-05 – 2026-07-11
- **Github Repository:** https://github.com/ndmhuy/SuperMarioGame.git

Tasks Completed This Week

25125083 - Nguyễn Đình Minh Huy (Member A - Engine, Physics & Player/ Item Entities)

Branch: `A/physics-input-test` ***(Physics Refactoring, Camera & Level System)***

- **Task 1: Full Camera Class Implementation**: Implemented the Camera class with smooth LERP coordinate tracking, random coordinate offsets for screen shake, and viewport clamping boundaries that center the screen when level sections are smaller than standard viewport dimensions (`1280x720`).
- **Task 2: Decoupled Physics Command Intents**: Refactored movement and sprint commands (`MoveLeftCommand` , `MoveRightCommand` , `RunCommand`) to register intent state flags (`m_moveLeftRequested` , `m_moveRightRequested` , `m_runRequested`) directly on the Character or Player instead of applying velocity modifications directly.
- **Task 3: Momentum and Friction Centralization**: Moved passive horizontal deceleration/ friction, active horizontal acceleration, and run speed limits into the central PhysicsEngine::update() pipeline. Surface-specific properties (such as low friction on Ice tiles) are read dynamically.
- **Task 4: Camera & Playground Integration**: Declared and initialized the `Camera` inside PlayingState. Updated the rendering loop to support dual-view drawing (world elements render in camera-relative space while HUD and developer tools render in default static screen space).

- **Task 5: Mid-Air Wall Friction & Sliding Mechanics**: Added slide friction inside tile resolution. When an airborne character slides against a solid tile horizontally, any upward momentum is zeroed out, and downward sliding speed is capped at `Constants::WALL_SLIDE_SPEED = 50.0f` px/s.
- **Task 6: JSON LevelLoader Integration**: Configured CMake to pull and build header-only `nlohmann/json` v3.11.3. Implemented LevelLoader to parse coordinate-based maps, load tile dimensions, player spawn locations, and spawn items dynamically from files.
- **Task 7: Level Design**: Authored 4 complete level files (`level_1.json` , `level_2.json` , `level_3.json` , `bonus_1.json`) representing Grassland, Cave, Castle, and Sky environments. Created an ImGui level loader selector for real-time playtesting.

25125084 - Trần Gia Huy (Member B - Input, Sound & Gameplay Systems)

Branch: ***`B/feature/entities`*** ***(Gameplay Entities, Blocks & Animation System)***

- **Task 1: Enemy AI Movement Strategies**: Defined the IMovementStrategy interface and implemented 8 concrete strategies (Patrol, Chase, Fly, TimerEmergence, Linear, HammerThrow, TetheredChase, ProximityTrigger) to decouple enemy AI movement from concrete entity classes.
- **Task 2: Concrete Base Enemy Implementations**: Coded the Enemy base class and derived types Goomba, KoopaTroopa, KoopaParatroopa, and Boo supporting color variants, stomping physics, shell projection, and target-tracking chases.
- **Task 3: Original Block Implementations**: Programmed the Block base class and derived blocks: BrickBlock (breakable, drops coins), QuestionBlock (dispenses items, sets to empty state), Pipe (warp functionality on pressing down), and Flagpole (completes level, calculates height-scaled scores).
- **Task 4: Advanced Enemy Implementations (v2.0)**: Created 7 new enemy types: PiranhaPlant, BulletBill, HammerBro, Thwomp, ChainChomp, Lakitu, and Spiny, supporting spawning projectiles, proximity triggers, stomping immunity, and tethered chases.
- **Task 5: Advanced Block Implementations (v2.0)**: Coded 5 new interactive blocks: HiddenBlock (invisible until bumped), MovingPlatform (translates and carries player), FallingPlatform (4-state shaking/falling sequence), IceBlock (reduces surface friction), and ConveyorBelt (constant horizontal lateral push).
- **Task 6: EntityFactory Integration**: Implemented EntityFactory to dynamically instantiate and configure all 25+ entity types from JSON coordinates and config descriptors (`entities.json`).
- **Task 7: SpriteSheet Atlas & Flyweight Animations**: Implemented SpriteSheet to load texture sheets, Animation (flyweight template containing texture coordinates/duration), and

AnimationManager / Animator tracking playback states across expanded player and enemy configurations.

- **Task 8: Unit Verification Testing**: Wrote test configurations and assertion harnesses (`verify_enemies_new.cpp`, `verify_blocks_new.cpp`) to test enemy behaviors, projectile spawning, platforms, and state transitions in isolation.

AI Usage Declaration

- Used AI to troubleshoot coordinate scaling issues between scrollable camera views and static screen space rendering.
- Used AI to design the internal edge collision algorithm that filters horizontal floor boundary crossings.
- Used AI to refine the timing sequence of `onGround` status resets inside the physics pipeline.
- Used AI to help structure the Flyweight pattern in the animation system and separate static animation metadata from dynamic animator state.
- Used AI to coordinate the state machine loops of the FallingPlatform and Thwomp entities.

Tasks Planned for Next Week

- **Nguyễn Đình Minh Huy**: Implement Phase 6 (Level Editor & UI overlays, Save/Load slot selector, etc.) and start Phase 7 (Game States & Game Over screen, World Map), and begin integrating advanced mechanics (Time Rewind, Shadow Mario).
- **Trần Gia Huy**: Implement Phase 5.3 (HUD, Minimap, Particles) and begin Phase 10 (Advanced Systems - Achievements, Stats, Console Debug commands) or Phase 6 level design additions.

Issues & Resolutions

Issue 1: Internal Edge Collision Bug (Stuck on Flat Ground)

- **Problem**: When walking across flat floors composed of adjacent tiles, the player would randomly stop or stick at tile seams. This occurred because horizontal overlaps at the

border were misidentified as horizontal side wall collisions (`normal.x != 0.0f`), which cancelled `velocity.x` and set `onWall = true` .

- **Resolution**: Added internal edge checking inside `CollisionDetector::checkEntityVsTileMap()` . If a horizontal collision is detected on a tile, we check the tile immediately above it. If that upper tile is empty/non-solid, we know this is a flat floor seam, not a wall, and dynamically convert it into a vertical Y push-out, preserving horizontal velocity.

Issue 2: Ground Flag Jittering & Stale Bounding Box Sync

- **Problem**: Standing or walking on tiles caused the `onGround` state to rapidly toggle between `TRUE` and `FALSE` between consecutive frames, causing the player to jitter slightly.
- **Resolution**: Found that when `resolveEntityVsTile` updated `entity.position` , it did not synchronize the `boundingBox` coordinates. Stale bounding boxes remained inside the tile on the next frame, re-triggering overlaps. Added immediate `boundingBox` synchronization steps upon any resolution displacement.

Issue 3: Double-Resolution Cumulative Push-outs

- **Problem**: When a player spanned two adjacent floor tiles, both floor tile collisions were resolved sequentially. The push-outs were summed together (e.g. pushed up twice), throwing the player slightly into the air, which immediately set `onGround = false` .
- **Resolution**: Refactored the Y and X collision resolution steps inside `PhysicsEngine::update()` to find and resolve only the single collision containing the **maximum overlap** (largest penetration depth) along each axis per frame. Resolving only the maximum overlap is mathematically sufficient to clear all overlaps on flat ground without double-counting pushes.

Issue 4: Wall Momentum Retention (Air Wall Collision)

- **Problem**: Jumping and holding movement keys against a vertical wall allowed the player to glide upwards along the wall with full momentum.
- **Resolution**: Added wall friction inside `resolveEntityVsTile()` . If a player collides horizontally with a wall in mid-air (`!onGround`), their upward vertical velocity (`velocity.y < 0.0f`) is zeroed out to simulate friction/momentum loss. Falling velocity is also capped at `WALL_SLIDE_SPEED = 50.0f` px/s.